UF–Core Kernel Security Specification v1.0

UF-Core / SES-Core Working Draft

March 8, 2026

1 Scope and Objectives

This specification defines *security requirements* for the UF–Core Kernel (L0–L4) as a reusable, domain-agnostic structural engine.

It is independent of any specific application (e.g. TFE), cloud provider, or device platform. It assumes a separate cryptographic platform (**SCE / SES–Core**) is available to provide key management, envelope protection, and attestation.

Normative Goals

The security objectives are:

- Protect the **UF Kernel implementation** (L0–L4) against practical reverse-engineering and tampering.
- Protect the **UF Structural Payload** (UF–SP) and any derivative representations from disclosure or undetected modification.
- Support **multi-tenant, multi-environment** deployment (cloud, edge, device) under a single, deterministic model.
- Provide a **Threat and Response Matrix** that defines mandatory behavior when specific security signals are observed.

All requirements in this document are **normative** unless explicitly marked as *optional profile*.

2 Terminology and Symbols

UF Kernel and Structures

- **UF Kernel**: Core structural engine, implementing L0–L4.
- **SEV**: Structural Energy Vector at L0.
- **Gate**: Segmented structural interval at L1.
- **DSF**: Decision State Field at L4.
- **UF–SP**: UF Structural Payload (full internal state).
- **SES Summary**: Redacted subset of UF–SP allowed in plaintext.

Metrics

For a UF Kernel evaluation over a window of K gates, define:

- $DSF_k = (D_k, M_k, R_k, U_k^*, C_k, P_k, B_k)$ for gate k.
- S(UF): global stability functional.
- R(UF): robustness index.
- GSS: gate stability score (scalar summary).
- DSS: directional stability score.

Crypto Platform

- **SES–Core**: Security Envelope System providing key hierarchy, domain binding, envelope format, and AEAD encryption.
- **SCE**: Structural Cryptography Engine implementing the cryptographic primitives required by SES–Core.
- K_{root}: root key stored in KMS/HSM or equivalent.
- K_{domain}: derived key bound to a specific UF domain.
- **DomainParams**: canonical tuple that defines a UF domain (symbol, window, timeframe, UF version, SES version, tenant, engine variant, and environment).

3 UF-Core Structural Metrics (Mathematical Summary)

3.1 Decision Surface Formalism

For each gate k the Decision State Field vector is:

$$DSF_k = (D_k, M_k, R_k, U_k^*, C_k, P_k, B_k)$$

where, informally:

- D_k = dominant direction component (e.g. up/down/flat).
- M_k = magnitude / strength.
- R_k = resonance component.
- U_k^* = normalized uncertainty (bounded in $[0, 1]$).
- C_k = collapse indicator (bounded above by $C_{\max}$).
- P_k = pressure / persistence term.
- B_k = breathing term (bounded in $[B_{\min}, B_{\max}]$).

3.2 Global Stability Functional $S(\mathrm{UF})$

The global stability functional aggregates gate-level invariants:

$$S(\mathrm{UF}) = \lambda_c(1 - \mathrm{Coh}) + \lambda_h(1 - H_s) + \lambda_b(1 - \mathrm{Var}(B)) + \lambda_u(1 - \bar{U}) + \lambda_d(1 - \mathrm{Drift}(T))$$

with:

- $\mathrm{Coh} \in [0, 1]$: coherence of DSF directions.
- H_s: structural entropy measure.
- $\mathrm{Var}(B)$: variance of breathing across gates.
- $\bar{U}$: average uncertainty across gates.
- $\mathrm{Drift}(T)$: normalized drift functional.
- $\lambda_c, \lambda_h, \lambda_b, \lambda_u, \lambda_d \geq 0$: fixed weights chosen at specification time.

3.3 Robustness Index $R(\mathrm{UF})$

Let $S^{(i)}(\mathrm{UF})$ denote the stability of UF under the i-th perturbation (e.g. rescaling, noise, time shift). The robustness index is:

$$R(\mathrm{UF}) = 1 - \frac{1}{K} \sum_{i=1}^{K} \left| S(\mathrm{UF}) - S^{(i)}(\mathrm{UF}) \right|$$

with invariants:

$$0 \leq R(\mathrm{UF}) \leq 1$$

A lower $R(\mathrm{UF})$ indicates high sensitivity to perturbation.

4 Threat Model

4.1 Adversarial Goals

The adversary may attempt to:

1. Recover internal UF structure (UF–SP).
2. Infer UF operators or internal dynamics.
3. Tamper with UF outputs or their integrity.
4. Manipulate domain binding parameters to replay or recontextualize UF outputs.
5. Clone, roll back, or substitute UF Kernel versions.

4.2 Out-of-scope Threats

The following are explicitly excluded from this specification:

- Full OS compromise or hypervisor compromise.
- KMS/HSM compromise.
- Physical hardware attacks and side-channel attacks.

5 Security Architecture Overview

UF-Core MUST be deployed inside a layered security architecture:

1. **Kernel Boundary**: clearly defined runtime boundary for L0–L4 and any internal state (UF–SP).
2. **Envelope Layer**: SES–Core/SCE protecting UF–SP and SES Summaries.
3. **Attestation Layer**: mechanisms that verify engine integrity and environment.
4. **Diversification Layer**: per-tenant / per-device variation to prevent one compromise from generalizing.
5. **Monitoring Layer**: chain-of-custody logging and anomaly detection.

All external systems (e.g. apps, devices) interact with UF-Core only through:

- **UF Engine API**: functional calls (e.g. evaluate_series) that accept raw series and return SES Summaries or envelopes.

6 UF Structural Payload and Redaction

6.1 UF Structural Payload (UF–SP)

UF–SP SHALL include at minimum:

- SEV sequences from L0.
- Gate boundaries and attributes from L1.
- Interpretive vectors from L2.
- Resonance curves from L3.
- DSF vectors from L4.
- Validation and hardening diagnostics.

UF–SP MUST be serialized deterministically (e.g. canonical JSON or an equivalent canonical binary form) prior to encryption.

6.2 Structural Redaction Policy

Plaintext outputs from the UF Engine boundary SHALL be limited to:

- S(UF).
- R(UF).
- GSS (Gate Stability Score).
- DSS (Directional Stability Score).
- DSF stability scalar.
- Collapse flags (boolean / categorical).
- SafeMode indicator.

All other fields in UF–SP MUST be encrypted inside a SES Envelope.

Application-level consumers MUST NOT receive raw SEV, gate internals, full DSF vectors, or per-gate diagnostics.

7 UF-Protect SCE Profile (Normative)

This section defines the *UF-Protect SCE Profile*: the concrete instantiation of SES–Core / SCE for UF-Core.

7.1 Domain Parameters

Define the canonical domain parameter tuple:

$$\text{DomainParams} = (\text{symbol}, \text{window\_start}, \text{window\_end}, \text{timeframe}, \text{uf\_version}, \text{ses\_version}, \text{tenant\_id}, \text{engine\_variant},$$

where:

- `symbol`: identifier (e.g. ticker, device ID, sensor ID).
- `window_start`, `window_end`: ISO 8601 timestamps.
- `timeframe`: sampling granularity (e.g. 1D, 1m, 1s).
- `uf_version`: UF Kernel version (e.g. v1.4.0).
- `ses_version`: SES-Core version (e.g. v2.0).
- `tenant_id`: unique tenant identifier.
- `engine_variant`: identifier for per-device variant (see diversification).
- `environment`: deployment environment label (e.g. “aws-prod”, “edge-hospital-01”).

7.2 Key Hierarchy

SCE / SES-Core MUST implement a two-layer key hierarchy:

- Root key K_{root} is stored in KMS/HSM and never leaves its secure boundary.
- For each domain, derive a domain key:

$$K_{\text{domain}} = \text{HKDF}(K_{\text{root}}, \text{DomainParams})$$

HKDF MUST be used with a fixed, documented salt and info structure, so that key derivation is deterministic and reproducible.

7.3 Envelope Format

A UF SES Envelope SHALL be structured as:

$$\begin{aligned}\text{Envelope} = \{ & \\ & \text{header} : \{\text{DomainParams}\}, \\ & \text{summary} : \{S, R, GSS, DSS, \text{DSFStability}, \text{collapse\_flags}, \text{safemode}\}, \\ & \text{ciphertext} : C \\ \}&\end{aligned}$$

where:

- $S = S(\text{UF})$, $R = R(\text{UF})$.
- `collapse_flags` encodes structural collapse indicators.
- `safemode` is a boolean indicating UF safe-mode.
- C is the authenticated ciphertext of UF–SP.

7.4 Encode / Decode Algorithms

Let Serialize$(\cdot)$ and Deserialize$(\cdot)$ be deterministic (canonical) serializers.

Encode

function UFE_Encode(UF_SP, DomainParams) :
  header $\leftarrow$ DomainParams
  summary $\leftarrow$ Redact(UF_SP)
  $K \leftarrow \text{HKDF}(K_{\text{root}}, \text{DomainParams})$
  $A \leftarrow$ Serialize(header, summary)
  $P \leftarrow$ Serialize(UF_SP)
  $C \leftarrow$ SIV_Encrypt(K, P, A)
  return $\{\text{header}, \text{summary}, C\}$

Decode

function UFE_Decode(Envelope, DomainParams) :
  $K \leftarrow \text{HKDF}(K_{\text{root}}, \text{DomainParams})$
  $A \leftarrow \text{Serialize}(\text{Envelope.header}, \text{Envelope.summary})$
  $P \leftarrow \text{SIV\_Decrypt}(K, \text{Envelope.ciphertext}, A)$
  UF_SP $\leftarrow$ Deserialize(P)
  return UF_SP

Decryption MUST fail if any element of `DomainParams` differs between encode and decode.

7.5 UF Engine Behavior

UF Kernel implementations MUST:

- Produce UF–SP internally and immediately wrap it using `UFE_Encode` before any persistence or transmission.
- Expose only SES Summaries (header + summary) to domain adapters unless explicit UF-SP access is granted under controlled conditions (e.g. offline validation).
- NEVER write UF–SP in plaintext to disk, logs, or network.

8 Deployment Profiles

8.1 Profile C: Cloud Service

Profile C defines UF-Core as a server-side engine in a cloud environment (e.g. AWS, GCP, Azure). Requirements:

- UF Kernel code and configuration reside in server-side repositories only; they MUST NOT be distributed to clients.
- All UF structural evaluations are executed inside controlled compute environments with:
  - secure boot or image signing,
  - restricted shell / debug access,
  - access to KMS/HSM for K_{root}.
- UF outputs persisted at rest MUST be SES Envelopes; plaintext summaries may be stored only if justified and documented.

8.2 Profile E: Edge / On-Prem Engine

Profile E defines UF-Core running on edge servers under operator control (e.g. hospital, factory). Requirements:

- Same encryption and envelope rules as Profile C.
- Optional use of hardware roots of trust (TPM, HSM) for K_{root}.
- Restricted local administrative interfaces.

8.3 Profile D: Constrained Device Wrapper

Profile D defines UF-Core not running natively on the device, but as a *wrapper service* (device → UF Engine service → device).

Requirements:

- Device sends raw measurement series to a UF Engine node.
- UF Engine evaluates series and returns only SES Summary or coarse decisions (e.g. ACCEPT/REJECT/ALERT).
- Device never receives raw UF–SP or Kernel code.

8.4 Optional Profile SHADOW: Shadow-UF Deployment

Profile SHADOW is an *optional* deployment where devices receive only a *mimic* of UF-Core:

- A learned model approximates UF behavior for a constrained domain.
- The true UF Kernel runs only in secure environments (Profile C/E).
- SHADOW engines are still protected with SES Envelopes, but even if fully extracted, they do not reveal the underlying UF math.

8.5 Optional Profile SPLIT: Split-Brain Deployment

Profile SPLIT is an *optional* deployment where the algorithm is divided:

- Device-side UF performs L0–L2 or partial DSF computation.
- Cloud-side UF completes L3–L4 and SES encoding.
- Reconstruction of full UF-Core requires both halves.

9 UF Threat & Response Matrix

This section is **normative**. For each threat, the specified responses MUST be implemented.

For brevity, we use:

- **Local Response**: behavior inside UF Kernel / UF Engine.
- **System Response**: behavior of the hosting system, service, or device.

Threat ID	Description	Detection Signals (UF / SES)	Required Response

T1	Code Extraction Attempt	UF Engine detects debug / introspection flags, breakpoint patterns, or execution under known analysis environments (where available).	Local: set `safemode=true`, restrict outputs to coarse SES Summaries, and disable UF–SP export. System: raise a security event in chain-of-custody log and notify operator.
T2	Code Tampering (on disk or in image)	Hash of UF Kernel bundle or manifest mismatch at startup or during periodic integrity check.	Local: refuse normal operation; enter "compromised" state; emit only explicit error codes. System: mark instance as untrusted, trigger replacement or quarantine, and route workloads away.
T3	UF–SP Exfiltration	Attempt to obtain raw UF–SP without SES Envelope, or Envelope with missing/invalid encryption fields.	Local: deny request, emit only SES Summary or error. System: log attempted exfiltration and optionally throttle or block caller.
T4	Envelope Replay / Misbinding	Decryption request with DomainParams that do not match Envelope header; SIV authentication failure.	Local: reject decryption; do NOT return UF–SP. System: log as possible replay attempt; optional rate-limiting or credential review.
T5	Engine Rollback / Clone	UF Engine runs with UF version or engine_variant inconsistent with expected deployment configuration for environment/tenant.	Local: flag configuration anomaly; optionally degrade outputs (safemode). System: prevent registration of mismatched versions and require explicit admin approval.
T6	Adversarial Environment	Runtime environment indicators show missing integrity guarantees (no secure boot, no KMS access where required, unknown host).	Local: restrict functionality to minimum safe mode; avoid processing sensitive data. System: deny registration of such nodes for production workloads.

T7	Rogue Tenant / Abuse of Interface	Excessive or abnormal query patterns for the same symbol/domain that indicate probing or brute-force attempts.	Local: enforce rate limits per tenant id and domain. System: throttle or suspend the tenant identity; raise operator alert.
T8	Domain Parameter Manipulation	Repeated attempts to alter symbol, timeframe, tenant_id, or environment to force cross-domain reuse.	Local: treat inconsistent DomainParams as error; never decrypt with mismatched parameters. System: flag suspicious parameter patterns for review.
T9	Data Integrity Tampering	Structural inconsistencies in input series (e.g. non-monotone timestamps, impossible prices) detected by UF validation.	Local: reject input; mark evaluation as invalid; do not emit normal SES Envelope. System: log anomaly and optionally request upstream data verification.
T10	Denial-of-Service via Collapse Induction	Repeated input patterns that drive UF into collapse or frequent SafeMode activation.	Local: limit repeated processing for identical or near-identical collapse-inducing inputs. System: apply stronger throttling, possibly blacklisting the source or isolating traffic.

10 Performance and Overhead Targets

Implementations SHOULD target the following overhead budgets relative to a baseline UF evaluation over a fixed window:

- Code integrity check at startup: $< 1\%$ additional latency.
- Periodic runtime integrity checks: amortized $< 2\%$ CPU overhead.
- SES encoding/decoding per UF evaluation: $< 15\%$ additional wall-clock time for typical UF–SP sizes.
- Attestation (remote challenge/response): tunable; SHOULD be executed out-of-band or at low duty cycle to avoid impacting real-time workloads.

These are targets, not hard limits, and MAY be tightened for high-value deployments.

11 Compliance Requirements

To claim conformance with this specification, an implementation MUST:

1. Implement UF-Core L0–L4 in a clearly defined Kernel boundary.
2. Implement SES–Core / SCE-based envelope protection over UF–SP according to the UF-Protect SCE Profile.
3. Ensure that all UF–SP at rest and in transit is encrypted and authenticated via SES Envelopes.
4. Restrict plaintext outputs to SES Summary fields only.
5. Implement all mandatory threat responses T1–T10 as defined in the Threat & Response Matrix.
6. Provide a chain-of-custody log that records:
   - UF Engine version and engine variant id,
   - SES version and key identifiers (non-sensitive),
   - domain parameters for each evaluation,
   - all security events (integrity failures, replays, abuse).

12 Glossary

UF-Core Unified Framework structural engine (L0–L4).

UF–SP UF Structural Payload; full internal structural state.

SES–Core Security Envelope System; key hierarchy and envelope protection layer.

SCE Structural Cryptography Engine; cryptographic implementation used by SES–Core.

SES Summary Whitelisted subset of UF–SP allowed in plaintext.

DomainParams Canonical tuple binding keys to a UF domain.

SES Envelope Container with header, summary, and ciphertext.

SafeMode UF state indicating structural instability or compromised environment; outputs are degraded or suppressed.

Engine Variant Identifier for a particular binary/implementation variant used for diversification.